

Imperfect GPS

Day 5 solution walkthrough

Competitive Programming Bootcamp

Statement excerpts:

- ▶ *“records its position periodically”*
- ▶ *“GPS can underestimate the total distance”*

Programming interpretation:

- ▶ Compute the true path length from every original segment.
- ▶ Compute the GPS path from sampled positions at fixed time intervals plus the final point.
- ▶ The output is the percentage of true distance lost by sampling.

Textbook connection: CP4 7.2.1 Points

Core technique: Geometry simulation with interpolation

- ▶ The actual path is the polyline through all recorded runner positions.
- ▶ The GPS samples the position every fixed interval and at the final time.
- ▶ The answer is the percentage distance lost by GPS sampling.

- ▶ Actual distance is a sum of Euclidean segment lengths.
- ▶ GPS samples may fall between known points.
- ▶ Constant speed on each segment allows linear interpolation.

- ▶ Compute `actual_distance` across consecutive input points.
- ▶ Start previous GPS point at the initial position.
- ▶ For each GPS time before the final time, find its containing segment.
- ▶ Interpolate `x` and `y` using the time ratio.
- ▶ Add GPS segment distances, then add the final position and compute percentage lost.

- ▶ `ratio = (gps_time - time1) / (time2 - time1).`
- ▶ `current = point1 + ratio * (point2 - point1).`
- ▶ `math.hypot` computes stable Euclidean distances.

- ▶ Actual distance sums exactly the straight segments described by the input path.
- ▶ Interpolation gives the runner's exact position at every GPS sample time under constant segment speed.
- ▶ The GPS distance is the polyline through those sampled positions, matching the device model.

Complexity: $O(N + T/\text{interval})$ time and $O(N)$ memory.

- ▶ The final GPS point is always included even when the final time is not a multiple of the interval.
- ▶ Use floating-point output; Kattis accepts tolerance.

Source lines: imperfectgps.py:46-64

```
import sys
from math import hypot

def distance(x1, y1, x2, y2):
    return hypot(x2 - x1, y2 - y1)

def main():
    input = sys.stdin.buffer.readline

    n, interval = map(int, input().split())

    points = []

    for _ in range(n):
        x, y, time = map(int, input().split())
        points.append((x, y, time))
```

What this chunk does

- ▶ hypot computes Euclidean segment lengths accurately.
- ▶ Each input point stores x, y, and timestamp.
- ▶ The interval controls when GPS readings are simulated.

Source lines: imperfectgps.py:66-76

```
actual_distance = 0.0

for i in range(1, n):
    x1, y1, _ = points[i - 1]
    x2, y2, _ = points[i]

    actual_distance += distance(x1, y1, x2, y2)
```

What this chunk does

- ▶ The real route follows every consecutive input point.
- ▶ Each segment length is added to `actual_distance`.
- ▶ This is the denominator for the final percentage loss.

Source lines: `imperfectgps.py:78-95`

```
previous_x = points[0][0]
previous_y = points[0][1]

gps_distance = 0.0

gps_time = interval

segment = 0

final_time = points[-1][2]
```

What this chunk does

- ▶ The GPS always records the starting position.
- ▶ `previous_x` and `previous_y` track the last recorded GPS point.
- ▶ `segment` identifies which true path segment contains the next GPS timestamp.

Source lines: imperfectgps.py:97-135

```
while gps_time < final_time:

    while points[segment + 1][2] < gps_time:
        segment += 1

    x1, y1, time1 = points[segment]
    x2, y2, time2 = points[segment + 1]

    ratio = (gps_time - time1) / (time2 - time1)

    current_x = x1 + ratio * (x2 - x1)
    current_y = y1 + ratio * (y2 - y1)

    gps_distance += distance(previous_x, previous_y, current_x, current_y)

    previous_x = current_x
    previous_y = current_y

    gps_time += interval
```

What this chunk does

- ▶ The loop processes each scheduled GPS time before the finish.
- ▶ The segment pointer advances until the timestamp lies in the correct true segment.
- ▶ Linear interpolation finds the runner's position at that time before adding GPS distance.

Source lines: imperfectgps.py:137-159

```
final_x = points[-1][0]
final_y = points[-1][1]

gps_distance += distance(previous_x, previous_y, final_x, final_y)

lost_distance = actual_distance - gps_distance

percentage_lost = (lost_distance / actual_distance) * 100

print(percentage_lost)

main()
```

What this chunk does

- ▶ The final position is always recorded, even off the regular interval.
- ▶ `lost_distance` compares actual path length with sampled GPS path length.
- ▶ The percentage is printed as a floating-point value.

Solution file: `practice_contests/day_5/imperfectgps.py`

Problem: <https://open.kattis.com/problems/imperfectgps>

Use this deck with the source code open beside it. The slides explain the reasoning; the code shows the exact input handling and output formatting.